

# Shiv Nadar University Chennai

|                     |                                               |             |
|---------------------|-----------------------------------------------|-------------|
| Degree & Branch     | B.Tech Artificial Intelligence & Data Science | Semester V  |
| Subject Code & Name | CS3801 – Web Technologies Lab                 | AY: 2026–27 |

## Assignment - 10 MERN Stack Todo Application

### 1 Aim:

To learn and implement the MERN stack and convert a previous React and Node.js project into a MERN-based application using React, Node.js, Express.js, and MongoDB.

### 2 Design:

The application follows a client-server architecture using the MERN stack. React is used to develop the frontend, Node.js and Express.js are used to create the backend REST API, and MongoDB is used to store the Todo data. Mongoose is used to connect the Node.js application with MongoDB.

### 3 Algorithm:

1. Start the application.
2. Initialize the React state with an empty `todos` array and an empty `newTodo` string.
3. Fetch the existing Todo tasks from the Express server when the React application starts.
4. Update the React state with the tasks received from the server.
5. Accept task input from the user through the Todo form.
6. Ignore the input if it is empty.
7. Create a new Todo object containing the task and completed status.
8. Send the new Todo to the Express server using a POST request.
9. Store the Todo in MongoDB using Mongoose.
10. Update the React state with the newly added Todo.
11. Allow the user to update the completed status using a PUT request.
12. Allow the user to delete a Todo using a DELETE request.
13. Display the updated list of Todo tasks.
14. Stop the application.

## 4 Code Implementation:

### 4.1 App.jsx

```
client > src > App.jsx > default
1  import { useEffect, useState } from "react";
2  import TaskForm from "../components/TaskForm";
3  import TaskList from "../components/TaskList";
4
5  function App() {
6    const [todos, setTodos] = useState([]);
7    const [newTodo, setNewTodo] = useState("");
8
9    const fetchTodos = async () => {
10     const response = await fetch("http://localhost:5000/api/todos");
11     const data = await response.json();
12     setTodos(data);
13   };
14
15   useEffect(() => {
16     fetchTodos();
17   }, []);
18
19   const addTodo = async () => {
20     if (newTodo.trim() === "") return;
21
22     const response = await fetch(
23       "http://localhost:5000/api/todos",
24       {
25         method: "POST",
26         headers: {
27           "Content-Type": "application/json"
28         },
29         body: JSON.stringify({
30           task: newTodo
31         })
32       }
33     );
34
35     const data = await response.json();
36
37     setTodos([...todos, data]);
38     setNewTodo("");
39   };
40
41   const toggleTodo = async (todo) => {
42     const response = await fetch(
43       "http://localhost:5000/api/todos/${todo._id}",
44       {
45         method: "PUT",
46         headers: {
47           "Content-Type": "application/json"
48         },
49         body: JSON.stringify({
50           completed: !todo.completed
51         })
52       }
53     );
54
55     const updatedTodo = await response.json();
56
57     setTodos(
58       todos.map((item) =>
59         item._id === updatedTodo._id
60           ? updatedTodo
61           : item
62       )
63     );
64   };
65
66   const deleteTodo = async (id) => {
67     await fetch(
68       "http://localhost:5000/api/todos/${id}",
69       {
70         method: "DELETE"
```

```

71     }
72   );
73
74   setTodos(
75     todos.filter((todo) => todo._id !== id)
76   );
77 };
78
79 return (
80   <div className="container">
81     <h1>MERN Todo Application</h1>
82
83     <p className="subtitle">
84       React + Express + MongoDB
85     </p>
86
87     <TaskForm
88       newTodo={newTodo}
89       setNewTodo={setNewTodo}
90       addTodo={addTodo}
91     />
92
93     <TaskList
94       todos={todos}
95       toggleTodo={toggleTodo}
96       deleteTodo={deleteTodo}
97     />
98   </div>
99 );
100 }
101
102 export default App;

```

## 4.2 TaskForm.jsx

```
client > src > components > TaskForm.jsx > default
1  function TaskForm({ newTodo, setNewTodo, addTodo }) {
2      const handleSubmit = (e) => {
3          e.preventDefault();
4
5          if (newTodo.trim() === "") {
6              return;
7          }
8
9          addTodo();
10     };
11
12     return (
13         <form onSubmit={handleSubmit}>
14             <input
15                 type="text"
16                 value={newTodo}
17                 onChange={(e) => setNewTodo(e.target.value)}
18                 placeholder="Enter a task"
19             />
20
21             <button type="submit">
22                 Add Task
23             </button>
24         </form>
25     );
26 }
27
28 export default TaskForm;
```

### 4.3 TaskList.jsx

```
client > src > components > TaskList.jsx > default
1  import TaskItem from "../TaskItem";
2
3  function TaskList({ todos, toggleTodo, deleteTodo }) {
4      return (
5          <ul>
6              {todos.map((todo) => (
7                  <TaskItem
8                      key={todo._id}
9                      todo={todo}
10                     toggleTodo={toggleTodo}
11                     deleteTodo={deleteTodo}
12                 />
13             ))}
14          </ul>
15      );
16  }
17
18  export default TaskList;
```

## 4.4 TaskItem.jsx

```
client > src > components > TaskItem.jsx > default
1  function TaskItem({ todo, toggleTodo, deleteTodo }) {
2      return (
3          <li>
4              <span
5                  onClick={() => toggleTodo(todo)}
6                  style={{
7                      textDecoration: todo.completed
8                          ? "line-through"
9                          : "none",
10                     cursor: "pointer"
11                 }}
12              >
13                  {todo.task}
14              </span>
15
16              <button onClick={() => deleteTodo(todo._id)}>
17                  Delete
18              </button>
19          </li>
20      );
21  }
22
23  export default TaskItem;
```

## 4.5 server.js

```
server > JS server.js > ...
1  const express = require("express");
2  const mongoose = require("mongoose");
3  const cors = require("cors");
4  require("dotenv").config();
5
6  const todoRoutes = require("./routes/todoRoutes");
7
8  const app = express();
9
10 const PORT = process.env.PORT || 5000;
11
12
13 // Middleware
14 app.use(cors());
15 app.use(express.json());
16
17
18 // MongoDB connection
19 mongoose.connect(process.env.MONGO_URI)
20   .then(() => {
21     console.log("MongoDB connected successfully!");
22   })
23   .catch((error) => {
24     console.error(
25       "MongoDB connection failed:",
26       error.message
27     );
28   });
29
30
31 // Routes
32 app.use("/api/todos", todoRoutes);
33
34
35 // Home route
36 app.get("/", (req, res) => {
37   res.send("MERN Todo Server is running!");
38 });
39
40
41
42
43 // Start server
44 app.listen(PORT, () => {
45   console.log(
46     `Server running at http://localhost:${PORT}`
47   );
48 });
49
50 });
```

## 4.6 todoRoutes.js

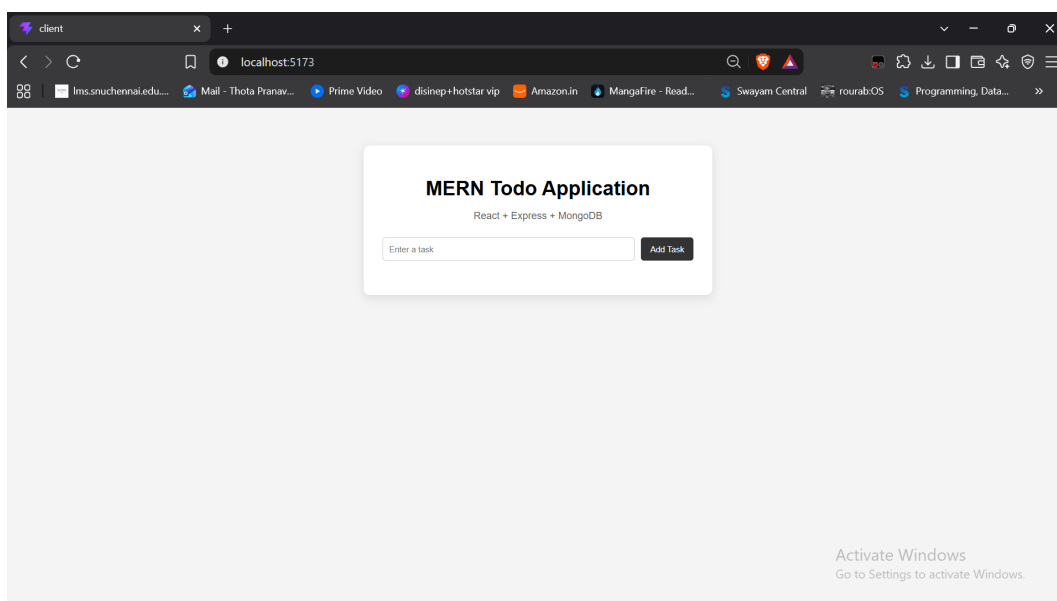
```
server > routes > JS todoRoutes > ...
1  const express = require("express");
2  const Todo = require("../models/Todo");
3  const router = express.Router();
4  // GET - Fetch all todos
5  router.get("/", async (req, res) => {
6    try {
7      const todos = await Todo.find();
8      res.json(todos);
9    } catch (error) {
10     res.status(500).json({
11       message: error.message
12     });
13   }
14 });
15 // POST - Add a new todo
16 router.post("/", async (req, res) => {
17   try {
18     const todo = new Todo({
19       task: req.body.task,
20       completed: false
21     });
22     const savedTodo = await todo.save();
23     res.status(201).json(savedTodo);
24   } catch (error) {
25     res.status(500).json({
26       message: error.message
27     });
28   }
29 }
30 });
31 // PUT - Update a todo
32 router.put("/:id", async (req, res) => {
33   try {
34     const updatedTodo = await Todo.findByIdAndUpdate(
35       req.params.id,
36       req.body,
37       {
38         new: true
39       }
40     );
41     res.json(updatedTodo);
42   } catch (error) {
43     res.status(500).json({
44       message: error.message
45     });
46   }
47 });
48 // DELETE - Delete a todo
49 router.delete("/:id", async (req, res) => {
50   try {
51     await Todo.findByIdAndDelete(req.params.id);
52     res.json({
53       message: "Todo deleted successfully"
54     });
55   } catch (error) {
56     res.status(500).json({
57       message: error.message
58     });
59   }
60 });
61 module.exports = router;
```

## 4.7 Todo.js

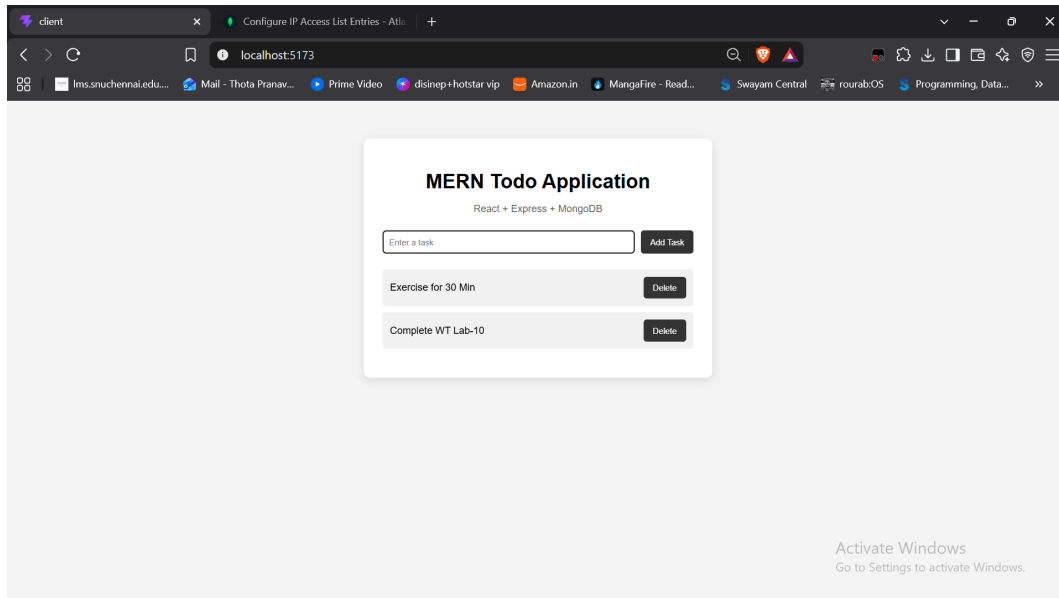
```
server > models > # Todo.js > _
1  const mongoose = require("mongoose");
2
3  const todoSchema = new mongoose.Schema({
4    task: {
5      type: String,
6      required: true
7    },
8
9    completed: {
10     type: Boolean,
11     default: false
12   }
13 });
14
15 module.exports = mongoose.model("Todo", todoSchema);
```

## 5 Web Page:

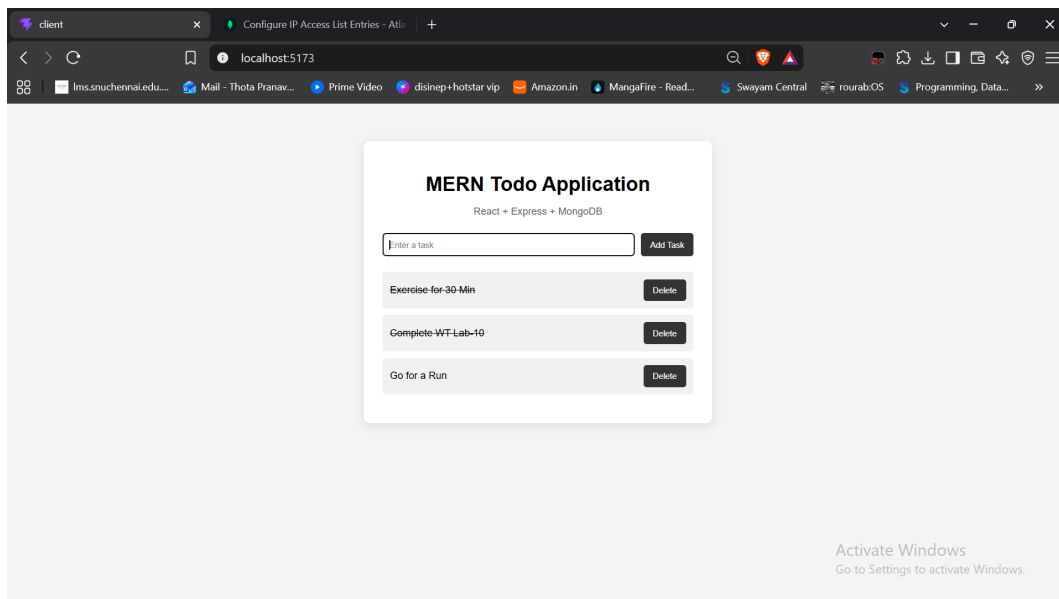
### 5.1 Todo Application:



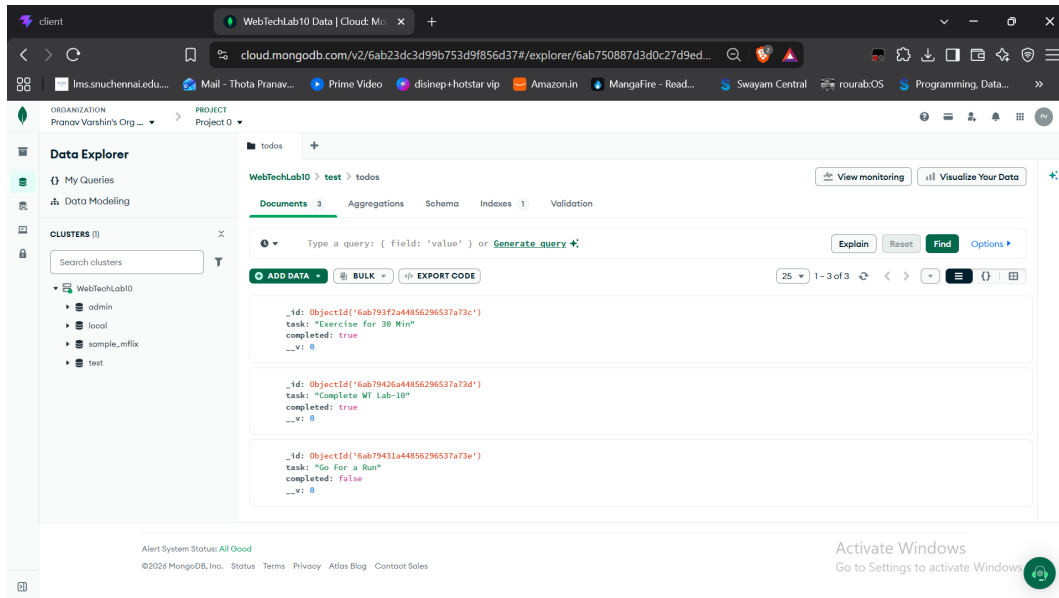
## 5.2 Adding Todo:



## 5.3 Completed Todo:



## 5.4 MongoDB Data:



## 6 Conclusion

The experiment successfully demonstrated the implementation of a MERN stack Todo application. React was used to develop the frontend components, Node.js and Express.js were used to create the RESTful backend API, and MongoDB was used to store Todo data. Mongoose was used for database interaction. The application successfully implemented CRUD operations for adding, viewing, updating, and deleting Todo tasks, demonstrating the integration of the complete MERN stack.

## 7 Code Link:

<https://github.com/PranavVarshin/Web-Technologies-Lab/tree/main/Lab-10>

- NAME: T Pranav Varshin(24110413)
- Reg No: 24011101116
- Section: AIDS-B